

1) 何时使用整体式体系结构

整体式应用程序最适合要求非常简单的用例，该应用程序应处理有限的流量。这方面的一个例子是组织的内部税计算应用程序或类似的开放公共工具。在这些用例中，企业确信用户群和流量不会随着时间的推移呈指数级增长。在某些情况下，开发团队决定从整体式体系结构开始，然后横向扩展到分布式微服务体系结构。

2) 何时使用微服务架构

微服务架构最适合复杂的用例，以及预计未来流量呈指数级增长的应用程序，如功能复杂的社交网络应用程序。典型的社交网络应用程序具有各种组件，如消息传递、实时聊天、实时视频流、图像上传、喜欢、共享功能等。在这种情况下，分别开发每个组件的同时需要牢记单一责任和关注点分离原则。选择整体式架构或微服务架构在很大程度上取决于实际的应用例。

16.2.3 Web应用架构模式

1. B/S 架构

B/S 架构的全称为 Browser/Server，即浏览器 / 服务器结构。Browser 指的是 Web 浏览器，极少数事务逻辑在前端实现，主要事务逻辑在服务器端实现。B/S 架构的系统无须特别安装，只需要有 Web 浏览器即可。该体系结构基于请求 - 响应模型。客户端将请求发送到服务器以获取信息，服务器对此进行响应。

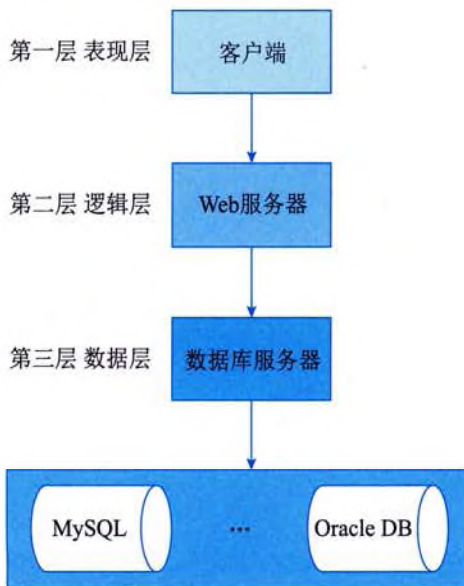


图 16-4 B/S 架构图

B/S 架构一般分为三层：第一层是表现层，主要完成用户和后台的交互及最终查询结果的输出功能；第二层是逻辑层，主要是利用服务器完成客户端的应用逻辑功能；第三层是数据层，主要是接受客户端请求后独立进行各种运算。B/S 的架构图如图 16-4 所示。

2. P2P 架构

P2P 架构是一种网络，是两个或多个客户端不经过服务器而直接通信的架构。可能首先会使用服务器，以让客户端间查找到对方，但在此之后就不再需要服务器了。具体分两种不同的形式：一种是完整连接拓扑架构，指的是每个客户端与其他客户端之间都必须有连接，信息可以直接在用户间交换；另一种是环形拓扑架构，指的是信息只有通过一个或多个客户端才能传递的架构。P2P 架构中的所有计

算机都具有相同的权限。因此，即使某些计算机 / 节点出现故障，网络和通信仍然处于正常运行状态。P2P 的基本架构如图 16-5 所示。

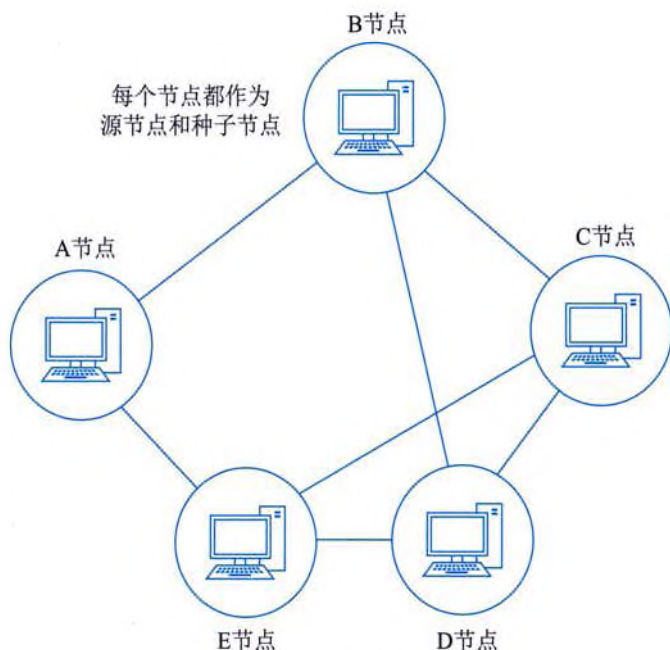


图 16-5 P2P 架构图

3. MVC/MVP 架构

MVC (Model-View-Controller) 体系结构是一种软件体系结构模式，其中应用程序逻辑根据功能分为三个组件。这些组件称为：

- 模型 (model)，表示数据在数据库中的存储方式；
- 视图 (view)，用户可见的组件，例如输出或 GUI；
- 控制器 (controller)，充当模型和视图之间接口的组件。

MVC 体系结构不仅用于桌面应用程序，还用于移动和 Web 应用程序。MVP (Model-View-Presenter) 模式是由 MVC 模式派生出的一种设计模式，其中 Model 提供数据，View 负责显示，Controller/Presenter 负责逻辑的处理。MVP 从 MVC 演变而来，通过表示器将视图与模型巧妙地分开。在该模式中，视图通常由表示器初始化，它呈现用户界面 (UI) 并接收用户所发出的命令，但不对用户的输入做任何逻辑处理，而是仅将用户输入转发给表示器。通常每个视图对应一个表示器，但是也可能一个拥有较复杂业务逻辑的视图会对应多个表示器，每个表示器完成该视图的一部分业务处理工作，降低了单个表示器的复杂程度，一个表示器也能被多个有着相同业务需求的视图复用，增加单个表示器的复用度。表示器包含大多数表示逻辑，用以处理视图，以及与模型交互以获取或更新数据等。模型描述了系统的处理逻辑，模型对于表示器和视图一无所知。在 MVP 里，Presenter 完全把 Model 和 View 进行了分离，主要的程序逻辑在 Presenter 里实现。Presenter 与具体的 View 是没有直接关联的，而是通过定义好的接口进行交互，从而使得在变更 View 时可以保持 Presenter 不变。

4. 微服务架构

微服务是由以单一应用程序构成的小服务，自己拥有自己的进程与轻量化处理，服务依据业务功能设计，以全自动的方式部署，与其他服务使用 HTTP API 通信。同时服务会使用最小规模的集中管理（如 Docker）能力，服务可以用不同的编程语言与数据库等组件实现。微服务架构区别于传统的单体软件架构，是一种为了适应当前互联网后台服务的三高需求：高并发、高性能、高可用而产生的软件架构。与微服务相对的另一个概念是传统的单体式应用程序（Monolithic Application），其内部包含了所有需要的服务。而且各个服务功能模块有很强的耦合性，彼此相互依赖，很难拆分和扩容。在微服务架构中，不同的功能 / 任务被拆分为单独的模块 / 代码库，这些模块 / 代码库相互协同工作，形成一个整体的大型服务。与单体架构相比，这种特殊的架构有助于更轻松、更干净的应用程序维护、功能开发、测试和部署。微服务的架构如图 16-6 所示。

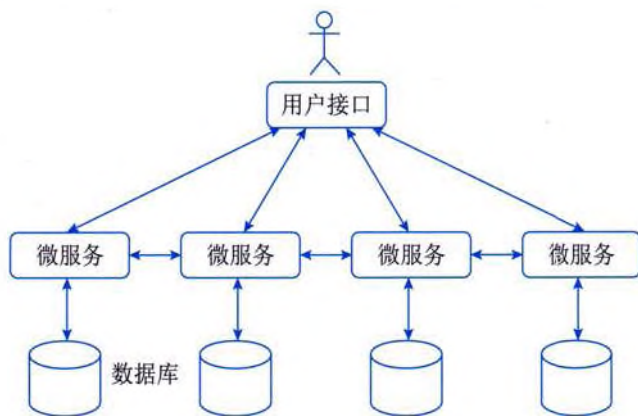


图 16-6 微服务结构图

微服务架构相对单体服务有很多优点，如技术异构性、隔离性、可扩展性、部署简单等。但这种架构也存在一些缺点。微服务把单体应用拆分成很多小的分布式服务，不可避免地会存在服务管理问题。为了克服微服务架构存在的缺点，一些技术概念相继被提出。

1) 服务注册与发现

微服务之间相互调用完成整体业务功能，如何在众多微服务中找到正确的目标服务地址，这就是所谓服务发现功能。常用的做法是服务提供方启动时把本服务的地址上报给服务注册中心，这就是服务注册。服务调用方订阅服务变更通知，动态接收服务注册中心推送的服务地址列表。

2) 服务监控

大型微服务架构的服务运维是一大挑战。服务运维人员需要实时掌握服务运行中的各种状态，一般需要控制面板监测服务的内存使用率、调用次数、健康状况等信息。这就需要系统有一套完备的服务监控体系，包括拓扑关系、监控（Metrics）、日志监控（Logging）、调用追踪（Trace）、告警通知、健康检查等。

3) 服务容错

任何服务都不能保证 100% 不出问题，生产环境复杂多变，服务运行过程中不可避免地会